FRE-GY-9743 Assignment 4: Futures Settlement and IFR Yield Curves

Raj Pawar, NYU Tandon School of Engineering, October 18, 2026.

Code: github.com/pawarraj8888/FRE-GY-9743-Assignments-4, branch `Raj_Pawar_Branch`. Project page: pawarraj8888.github.io/FRE-GY-9743-Assignments-4.

Part I: why the OTC price carries a discount factor and the futures price does not

Both trades move cash every day, and in both cases the amount is the same: today's mark-to-market minus what has already been posted. The difference is what that cash *is*.

Under a CSA the daily transfer is collateral, not settlement. The swap or forward is still a claim on its payoff $V(T)$ at T. What I post or receive each day is the other side's money parked against an unrealised mark: whoever holds it pays interest on it at the collateral rate r_C, and it flows back when the mark reverses or the trade matures. Nothing about the payoff has been paid early, so the value is still the present value of $V(T)$, and under perfect collateralisation the notes' equation (16) gives

$$V(t) = \mathbb{E}_t^Q\Big[e^{-\int_t^T r_C(u)\,du}\,V(T)\Big].$$

The intuition I find clearest: posting $V(t)$ as collateral is the same as being lent $V(t)$ today against $V(T)$ at maturity, and a loan from t to T at r_C is exactly the factor $e^{-\int_t^T r_C}$. The daily collateral calls do not change *when* the payoff is realised; they only make sure the money is there if someone defaults.

On the exchange the daily transfer is a settlement. At the close the clearing house pays me $F(t_{k+1},T) - F(t_k,T)$ outright, pays no interest on it, and re-strikes my contract at the new price so that it is worth exactly zero again. Nothing is ever owed for longer than one margining period. Writing the zero-value condition for that single period, with the overnight rate for the day fixed at its start so that $D(t_k,t_{k+1})$ is known at t_k,

$$0 = \mathbb{E}_{t_k}^Q\big[D(t_k,t_{k+1})\,(F(t_{k+1},T) - F(t_k,T))\big] = D(t_k,t_{k+1})\big(\mathbb{E}_{t_k}^Q[F(t_{k+1},T)] - F(t_k,T)\big),$$

so the discount factor drops out, $F(t_k,T) = \mathbb{E}_{t_k}^Q[F(t_{k+1},T)]$, and chaining the days gives $F(t,T) = \mathbb{E}_t^Q[S(T)]$. The discounting has not disappeared; it is applied one day at a time, over a horizon so short that the rate is known, and each day's bet is fully paid before the next day's rate is set. A one-day bet that must have zero value is priced the same whatever today's overnight rate is, because the rate multiplies both sides of the equation. The cumulative variation margin over the life is $F(T,T) - F(t,T) = S(T) - F(t,T)$, i.e. the forward-style payoff received in instalments, which is why no further time value of money appears in the price.

Same cash, different ownership. The exchange does not pay interest on my variation margin because the margin is mine: it is realised profit. The OTC counterparty pays interest on the collateral because the collateral is theirs: it secures an unrealised mark. The discount factor shows up exactly where interest is paid on the posted amount. The cleared-swap market made this explicit when it moved from collateralised-to-market (interest, called price alignment interest, paid on variation margin at r_C) to settled-to-market, where variation margin is a final settlement and the swap prices like a future.

What it changes. With a horizon-long, random discount factor the OTC forward price is a T-forward-measure expectation, $F^{\text{fwd}}(t,T) = \mathbb{E}_t^T[S(T)]$ (notes, section 2.3), while the futures price is $\mathbb{E}_t^Q[S(T)]$; the gap is the convexity adjustment, which comes from the covariance between the discount factor and the payoff. If rates were deterministic the discount factor could be taken out of the OTC expectation too and the two prices would coincide.

Part II: computational part

I completed the two `TODO` blocks in `fixedincomelib/yield_curve/yield_curve_model.py`. The component discount factor is $D_{\text{local}}(t) = \exp\big(-\int_0^t f(s)\,ds\big)$, with the integral taken from the component's piecewise-constant interpolator over curve time measured in ACT/ACT (ISDA) from the value date. The model's discount factor follows the registered reference recursively: a component without a reference is its own base (reference factor 1), and a funding component returns $D_{\text{ref}}(t)\,D_{\text{local}}(t)$, so `SOFR-1B-FLAT` is `SOFR-1B` times its own spread factor. The interpolator from Assignment 1 is carried over into the identical skeleton in `fixedincomelib/utilities/numerics.py`. In the notebook I created the 5-year `ProductFixedAccruedCashflow` through `qfCreateProductFixedAccruedCashflow` with payment on the termination date, discounted its payment c = notional × accrued on `SOFR-1B-FLAT`, and computed the analytic IFR risk with the chain rule $\partial V/\partial f_i = -V\,w_i$, where $w_i = \partial(\int_0^T f)/\partial f_i$ comes from `gradient_of_integrated_value_wrt_ordinate(0,T)` for both components. The notebook's own reference outputs are reproduced exactly.

Tenor	Date	Curve time	$D_{\text{SOFR-1B}}$	$D_{\text{SOFR-1B-FLAT}}$
6M	2027-04-07	0.498630	0.985152	0.985152
1Y	2027-10-07	1.000000	0.970446	0.970446
2Y	2028-10-10	2.008840	0.940565	0.940565
5Y	2031-10-07	5.000000	0.852156	0.852156
10Y	2036-10-07	10.000644	0.708223	0.708223
15Y	2041-10-07	15.000000	0.582765	0.582765

With the IFR grid $0.030, 0.031, \ldots, 0.039$ on the 1Y to 10Y nodes and a zero funding spread, the two curves agree. The 5-year cashflow pays on 2031-10-07: $c = 507.222222$ (ACT/360), $D = 0.852156$, $V = 432.232456$. Its risk to the first five SOFR nodes is $(-432.2325,\ -436.0536,\ -430.7797,\ -429.8641,\ -432.2325)$ per unit of IFR, i.e. $-V$ times the length of each node's bucket inside $[0, T]$ with $T = 5.000000$; nodes beyond five years carry zero risk, and the single funding node carries $-TV = -2161.162279$. `tests/test_yield_curve.py` (14 tests) checks the discount factors against a by-hand integral of the step function, flat extrapolation past the last node, the reference recursion with zero, flat and multi-node funding spreads, the notebook's numbers, the analytic risk against central bump-and-reval through `perturb_model_parameter`, serialization and the validation errors; `tests/test_interpolator_pcp.py` (20 tests) covers the interpolator.